

Introduction to R workshop

2026-06-26

1. Getting Started

R is a programming language – in this workshop we'll use RStudio as an interface to explore data, write some code, and visualize results!

You can also use the cloud version for free - <https://posit.cloud/plans/free>.

Install R and RStudio

First, download both R (cran.r-project.org) and RStudio (posit.co/downloads/). Go to <https://rstudio-education.github.io/hopr/starting.html> for more information.

This document

Within the code chunks, complete the lines of code requested. When you're ready to evaluate your code, set each chunk to evaluate (`eval = TRUE`) – 'knit' the document to create a nice pdf output for the entire document, or you can run specific code chunks by pressing the green triangle on the upper right hand side of that code chunk.

2. The Basics

Running code: console vs. script file

Navigate to the console in the bottom left of RStudio. Next to the `>` symbol, type `2*8` and press enter or return.

Now try running:

```
5 + "hello"
```

Your running history and the output for the code you run will show up in the console.

To save lines of code (or run multiple lines), use a script file!

Creating variables

We use the `<-` operator to assign values to variables in R.

1. Create a variable `my_age` in the code chunk below

```
my_age <-
```

2. Create a numeric vector `my_numbers` with three numbers separated by commas

```
my_numbers <- c(, , )
```

3. Create a variable that is equal to a word using “ ”
4. Update `my_age` by adding 5 years

Variables and data classes

Variables can be a lot of different things in R! Unlike some other languages, there is no need to initialize variables before you assign values. To see what R is classifying a specific variable (or structure of data) as, use the `class()` function.

1. Identify the data classes of the variables you just created

```
class(variable1)
class(variable2)
class(variable3)
```

Vectors

Earlier, we created a variable called `my_numbers` that contains multiple numbers – this is called a “vector” in R, which is a structure of data (rather than a data class itself).

We used the `c()` function to combine different numbers into this vector structure.

1. Combine different words into a vector

```
vector_of_words <- c()
```

Summary statistics

Now that we have a handful of values (in our numeric vector), we can do some summary statistics! We’ll use a few base R functions:

1. Find the `mean()`
2. Find the `sum()`
3. Find the `length()`

Indexing and logical filtering

There are often cases where we want to select a specific section of our data. In R, counting positions of a data structure is called indexing and choosing a specific piece of the data is finding a subsection or filtering the dataset.

1. Find the second element of your `my_numbers` vector using `[]`
2. Pull out the 6th and 7th elements of this vector using one line of code

```
example_vector <- sample(1:100, size = 10, replace = TRUE)
```

3. Pull out the values of `example_vector` that are greater than or equal to 10
4. Save the result of #3 in a new variable

3. Dataset example

Take a look at the documentation on the `beaver2` dataset in base R - note what the columns are named, the variables included, and the overall design of the study.

```
?beaver2
```

Exploring an example dataset

You can use the `head()` function to see the first 6 rows of the dataset. Each row in the dataset is a measurement of body temperature (the entire dataset is one individual beaver), and the columns correspond to the time (`day` and `time`) of data collection, the response (`temp`), and an additional potentially explanatory variable (`activ` - values are 0 (no activity outside retreat) or 1 (activity outside retreat)).

```
head(beaver2)
```

Another few great exploratory functions for your dataset:

```
str(beaver2) # "structure"  
summary(beaver2)
```

What is the average body temperature? What is the range of body temperatures?

Subset the dataframe

We know that the `activ` column indicates whether there was activity outside of a retreat, so perhaps we want to separate temperature measurements based on this activity. Figuring out how to subset our dataframe is useful here.

The `$` operator can be used to specify the name of a column in a dataframe when subsetting data.

1. Create a new variable that is our dataset of rows only where the beaver had activity outside the retreat

Don't forget the comma in your square brackets!

```
beaver_activity <- beaver2[, ]
```

2. Create a new variable that is our dataset of rows only where the beaver had a body temperature greater than 37.5 C

```
beaver_warm <-
```

3. Combine the two conditions mentioned above for a new variable called `active_warm`

Base R visualizations

Visualizing our data can help us understand the structure even further.

1. Histogram of body temperatures

```
hist(beaver2$temp)
```

Now we'll use the `boxplot()` function to look at the spread of body temperatures (`temp`) by detected activity (`activ`).

2. Replace the `y_value` and `x_value` elements with the appropriate column names

```
boxplot(y_value ~ x_value, data = beaver2,  
        xlab = "Activity (0 = none detected, 1 = detected)",  
        ylab = "Temperature",  
        main = "This is a title")
```

3. At what time does the beaver reach its highest temperature? Why is the code below wrong?

```
beaver2[max(beaver2$temp), ] #not correct!  
max(beaver2$temp) #returns maximum temperature value
```

Try again with `which.max()`.

More visualization

Here, we can use the graphing function included in the documentation to look at body temperature (`temp`) over the time of the study (progressive hours calculated using `day` and `time`). This function also includes a line showing presumably the high end of normal body temp (the `abline` line) and circles points where the beaver is noted as active (red circles, `points` line – `col = 2` corresponds to red).

```
beaver.plot <- function(bdat, ...) {  
  nam <- deparse(substitute(bdat))  
  with(bdat, {  
    # Hours since start of day:  
    hours <- time %/% 100 + 24*(day - day[1]) + (time %/% 100)/60  
    plot(hours, temp, type = "l", ...,  
          main = paste(nam, "body temperature"))  
    abline(h = 37.5, col = "gray", lty = 2)  
    is.act <- activ == 1  
    points(hours[is.act], temp[is.act], col = 2, cex = .8)  
  })  
}
```

```
beaver.plot(beaver2) #calling the function we just wrote
```

What trends of body temperature do you notice based on activity?

4. Writing functions

Functions are useful to write when we've copied and pasted code a few times!

1. Add a default value to the following function:

```
celsius_to_kelvin <- function(temp_C) {  
  temp_K <- temp_C + 273.15  
  return(temp_K)  
}  
celsius_to_kelvin()
```

2. Why is there an error message when we ask R to print `vect1` to the console?

```
celsius_to_kelvin <-function(temp_C) {  
  temp_K <- temp_C + 273.15  
  vect1 <- 1  
  return(temp_K)  
}  
  
vect1
```

3. Write a function using the `celsius_to_kelvin` function to convert a user-input value from fahrenheit to kelvin

5. Working with the tidyverse

Installing packages:

```
#install.packages("tidyverse")  
library(tidyverse)
```

tidyverse grammar

```
beaver2 %>%  
  filter(activ == 1) %>%  
  summarize(mean_temp = mean(temp))
```

1. Use the tidyverse to pull out star wars characters taller than 100 units (height) and of those characters, count the different species (species)

```
starwars %>%  
  filter(_____) %>%  
  count(_____)
```

ggplot2 graphing example

```
library(ggplot2)  
ggplot(beaver2,  
  aes(x = time, y = temp, color = activ)) +  
  geom_point()
```